LibGLP-1: Symbol-Level Debloating for Shared Libraries

***Abstract*—Most libraries used in the development of computing systems (even embedded) contain dead code and unused data. This exposes an increased attack surface since such code/data could include vectors that could be used to compromise the library, applications or even the system. Bloated, shared library code also increases the memory footprint of the system — a particular problem in the embedded domain.**

In this paper we focus on "*debloating*" libraries, a process that only retains code and data symbols that are necessary for the specific applications in the system. We introduce LibGLP-1, a framework that uses *symbol-level debloating* methods to achieve this objective. LibGLP-1 uses only static techniques and operates directly on ELF binaries — bypassing the need for library source code. Our evaluation shows that LibGLP-1 reduces individual libraries (libc, `librealsense`) by $70-77\%$, is able to significantly reduce microkernels and consistently achieves greater reductions than the state of the art (SOTA). In addition, LibGLP-1 is the first debloating tool that supports correct execution of shared libraries on embedded systems without a dynamic loader.

I. Introduction

Resource-constrained embedded systems are deployed at massive scale across IoT, automotive, industrial, and consumer devices. In these settings, exceeding memory budgets is either not an option on already deployed devices, or can force large hardware redesign costs such as requiring more hardware DRAM. With the rapid growth of AI workloads where more memory often translates to more capable systems, thus there's an increasing pressure on memory capacity. Broader industry demand for memory capacity can increase its cost [1]. This makes reducing software memory footprint both an engineering and economic priority.

Libraries represent a significant source of memory consumption. UNIX systems and development environments provide both shared libraries (otherwise known as dynamic libraries) and static libraries. Both aim to save memory by making different trade-offs. Shared libraries (`.so` files) are loaded at `exec` time by the dynamic loader (`ld`), or manually via the `dlopen` APIs. When a given shared library is loaded into multiple programs, the dynamic loader ensures that all read-only memory sections in the library (including executable code) is aliased. This ensures that they avoid memory replication across binaries and keep only a single copy of those sections in memory. A shared library includes multiple object files and only those that are required by the application are used. Thus, shared libraries include only the memory object files that are needed. In contrast, static libraries (`.a` files) are compiled directly into the applications that link with them.

Note that both of these traditional approaches are limited. Shared libraries must include all of a library's code and data, even if applications don't use most of it. In contrast, static libraries include object files that must be replicated across different applications that use the same library.

It is natural to ask if it is possible to get the best of both approaches as depicted in Figure 1. Can we pare down shared libraries to include only those functions and data required by some application on the system, while maintaining the ability to share the read-only data and executable code across applications? Toward this, previous research makes a "closed world" assumption: that we know all software required for an embedded system a-priori. This assumption is natural in many embedded environments. Yocto [2] and Buildroot [3] Linux include a manifest of all software on the system, Autosar variants require a specification of all applications for a system image, while component-based systems [4] and microkernels [5] specify and build protection domains for specific applications and servers separated with capabilities. Event containers [6] are often self-contained, including a static set of applications. Given this motivation, `ELFTailor` [7], `D-Linker` [8], and `Software Multiplexing` [9] all attempt to customize the contents of libraries for the specific set of applications that require them, using a combination of static and dynamic analysis.

Unfortunately, these approaches make assumptions that are not ideal for embedded systems. The granularity they use to remove code and data is coarse grained – either code pages (`ELFTailor`) or object files (`D-Linker`). Both use *dynamic analysis* of application and library execution to understand the reachable code and data. By eliminating memory on a coarse-grained basis, these approaches are not *complete* – some memory that is not used is not eliminated. Approaches that depend on a dynamic analysis require tests or applications that are *guaranteed* to exercise all code paths, otherwise they are not *sound* and could cause faults at runtime – where code is removed as it is not discovered by a dynamic analysis, but it is required once deployed.

Of late, "debloating" approaches [10] have become common; they (automatically, or even manually) reduce the unused code used in a system. The main idea is that a reduction in unused code or data *reduces the attack surfaces* available to an adversary, removes bugs and vulnerabilities (both, known and unknown) and demonstrates memory savings and performance improvements [11]. Debloating has been applied to containers [12], operating systems kernels [11], [13] and even libraries. These prior works have shown significant improvements and elimination of many (known) CVEs [11]. One of the challenges with directly applying these debloating methods is that they often require source code and even configuration files — which may be difficult to obtain in many environments. For example, among other proprietary tools, firmware/libraries for localization hardware (e.g., Intel Realsense, VICON Tracker) come pre-compiled. In this paper, we intend to leverage the debloating concepts and techniques while applying them *directly to ELF objects*, thus bypassing the aforementioned constraints.

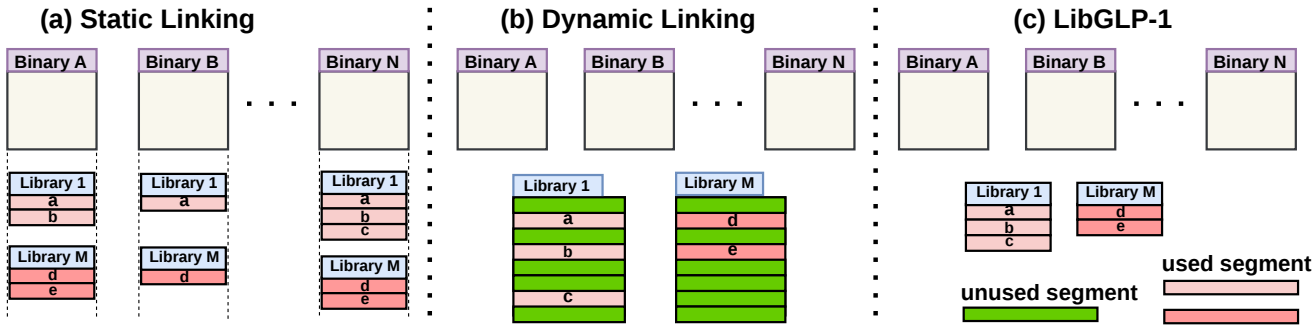


Fig. 1: Code and data using static linking, dynamic linking, and LibGLP-1. (a) Static linking copies only the utilized library objects into an ELF binary, but duplicates objects across binaries. (b) Dynamic linking allows multiple ELF binaries to share a single general-purpose shared library (avoiding per-binary duplication), but the shared library includes unused functions/data. (c) LibGLP-1 debloats the shared library to retain only the functions/data required by the target ELF binary(ies).

LibGLP-1 leverages the closed-world assumption and represents a sound and complete approach that will eliminate *all* library code and data that is not used by any application that relies only on static analysis, while enabling (as shared objects) code and read-only data sharing across processes. It depends only on the analysis and manipulation of ELF objects, thus avoids binary disassembly and analysis – which adds complexity and has fundamental limitations [14], the analysis of source – which cannot work on proprietary codes and binds the analysis to a language, or relying on compiler IR analysis – which binds the results to a specific compiler and requires access beyond the binary. LibGLP-1 requires no runtime support (*i.e.* it uses the default loader), thus does not negatively impact the predictability properties of execution nor performance.

LibGLP-1 consists of three major components. A *binary shrinker* computes the required symbols (*e.g.* variables, functions) using a reachability analysis across the closed-world set of application executables and rebuilds the shared libraries to completely eliminate unused functions and data. This technique depends on ELF "sections" separately wrapping each function and variable in the object. While these can be inserted at compile-time, LibGLP-1 also includes a *sectionifier* program that takes an object and provides a version with each function and variable wrapped in sections. This enables the binary shrinker to be used directly on existing ELF objects. Last, LibGLP-1 supports embedded systems that do *not* support a dynamic loader. It does this with a *relocation planner* that enables the correct binding and relocation of shared-library references without a generic loader. This enables a further debloating of the system by removing the traditionally complex dynamic loader.

The major contributions of LibGLP-1 are:

- **Shared-library minimization:** preserves shared-library memory reuse while statically enabling sound and complete function and data-level elimination for closed-world workloads.
- **Runtime-free deployment:** a build-time relocation planner and lightweight load-time that replaces the complex `ld.so`, yet performs full binding and relocations.
- **Configurable specialization scope:** supports per-workload sharing or per-application isolation to trade memory sharing against attack-surface reduction.
- **Evaluation:** We demonstrate that LibGLP-1 can significantly improve security properties and decrease memory consumption compared to `Piece-wise` [15], `ELFTailor` [7], and `D-Linker` [8] on a variety of applications including an autonomous vehicle stack. Additionally, we show the utility of the system in a component-based system that does not support a full loader.

II. Background

ELF is the standard binary format on Linux, and is widely used on embedded systems. To debloat library objects, LibGLP-1 relies on multiple ELF file structures (sections, segments, symbols, relocations) and standard dynamic-linking conventions (the GOT) to remove code and data while preserving correct linking and loading. In doing so, it avoids requiring source code, and avoids analysing the binary code itself. This object-file-centric approach enables the shrinking of proprietary code in which only the object is available, and avoids error-prone [14], [16] code decompilation.

A. ELF Fundamentals

Sections. ELF files organize program contents into multiple sections. The `.text` section contains executable machine code. The `.rodata` section contains read-only constants such as string literals. The `.data` section contains initialized global and static variables. The `.bss` section represents uninitialized global and static variables; it contributes to the in-memory footprint but typically does not occupy file bytes. ELF also stores relocation information in sections named `.rel.*` and `.rela.*`, which describe address fixups applied by the linker and/or the runtime loader. A section is the smallest unit that the linker can keep or discard. If each symbol is placed in its own section, section-level retention and elimination become equivalent to symbol-level retention and elimination, thereby enabling symbol-level debloating.

Segments. While sections describe how the linker views and manipulates the file, the OS maps ELF binaries into memory using segments specified by the program header

table. A read–execute (RX) loadable segment maps executable code including the `.text` section. A read-only (R) loadable segment maps read-only data e.g., `.rodata`. A read–write (RW) loadable segment maps writable globals such as `.data` and reserves space for the zero-initialized region `.bss`.

Symbols. ELF represents functions and data objects as symbols, each with a name, type (e.g., function or data), binding (local or global), and a definition location. Symbols may be defined (provided by the object) or undefined (referenced but provided elsewhere). During static linking, the linker resolves undefined symbols across all input objects to produce the final executable or shared object. During dynamic linking, executables and shared libraries export global symbols and import undefined symbols, which are resolved by the loader.

Relocation. At build time, the final addresses of some symbols are not yet known. Therefore, references to those symbols in code/data cannot yet be fully resolved. Relocation is the process by which the linker or loader updates the code or data locations containing those references with the correct address or offset. To do this, the linker/loader consults relocation entries, each of which specifies the relocation site, the referenced symbol, and the relocation type.

Global Offset Table Position-independent code (PIC) avoids hard-coded absolute addresses in instructions, so a binary built with PIC can execute correctly regardless of where it is loaded in memory. However, some symbol accesses still require an actual absolute address at runtime. Therefore, that absolute address must be stored somewhere else. The Global Offset Table (GOT) provides that storage by holding the absolute address in the corresponding GOT entry. The binary can then access the symbol indirectly through that entry.

B. Fine-Grained Sections and Link-Time Garbage Collection

Relocation, and potentially debloating away code or data might be performed at the section level. By default, functions compiled within the same translation unit (*i.e.* the same C file) may be placed into a single `.text` section of the resulting relocatable object, and many global objects may be aggregated into `.rodata` or `.data`. As a result, retaining one referenced function or object can implicitly retain other unused functions or data colocated in the same section, limiting the effectiveness of link-time elimination.

To enable finer-grained removal, modern toolchains provide options that place each function and each data object into its own section. The compiler flags `-ffunction-sections` and `-fdata-sections` instruct the compiler to emit one section per function and one section per data object, respectively. This layout typically produces section names such as `.text.<func>` and `.rodata.<obj>`/`.data.<obj>`, allowing the linker to reason about individual functions and objects rather than entire translation units.

Given section-level granularity, modern linkers can perform *section garbage collection* to remove unreachable code and data. In GNU `ld` and `gold`, this optimization is enabled via `--gc-sections`. Conceptually, the linker constructs a reachability graph rooted at required entry points and referenced sections, and discards sections that are not reachable from these roots. This mechanism is commonly used to reduce binary size in embedded systems, and it provides a key building block for fine-grained debloating of libraries.

However, this method does not, for example, apply to shared libraries for which reachability should be defined relative to applications and other libraries. As such, it is often used by applications, but is less usefully applied to libraries. Further, most existing binaries are not generated with variable-per-section and function-per-section. LibGLP-1 alleviates these shortcomings by creating mechanisms to provide function- and variable-per-section even if binaries aren't compiled as such, and by applying closed-world garbage collection on the code and data in shared libraries.

III. Design

Dynamic linking often increases memory footprint and attack surfaces by including code and data from shared-libraries — code and data that may never be used by the application. Prior debloating systems [7], [15], [17], [18], [19] attempt to reduce these costs by eliminating unnecessary functionality, often using information available in the source code or the program binaries. However, the information available may be not be sufficient for full static analysis (to identify *exactly* what parts of the system need to be debloated), so prior systems often rely on runtime tracing to improve dependency recovery [7], [15]. While other work [8] demonstrates dependencies can be recovered conservatively via static analysis, there is a reliance on dynamic tracing to improve precision. Moreover, removal typically remains limited to the granularity of eliminating entire object files; there is no option to include some functions from a library while debloating others. LibGLP-1, in contrast, while operating at the object-file level, *debloats shared libraries through fully static analysis and symbol-granularity removal*. Figure 2 presents an overview of LibGLP-1. Blue boxes denote inputs, purple boxes denote the main components of LibGLP-1, and red boxes denote outputs. Yellow labels indicate the sections that describe each component in detail.

The main design component of LibGLP-1 is binary shrinker. Figure 3 illustrates the workflow of LibGLP-1's binary shrinker. In this workflow, static analysis identifies the functions and data that must be retained, while relinking removes unnecessary symbols. The following subsections describe the static analysis and relinking in detail. The last subsections describe the relocation planner.

A. Static Analysis

The static analysis in LibGLP-1 relies on relocatable ELF objects to generate correct undefined symbol set – those that aren't stripped and maintain relocation entries. Prior binary- and decompilation-based approaches often struggle with constructs such as function pointers, callback tables, and C++ virtual dispatch, because the required dependency edges are not always explicit in the final binary and must instead be inferred from runtime. For example, a function call might appear as a `call *%rax` in the code, and to understand the function dependency being called, a complete

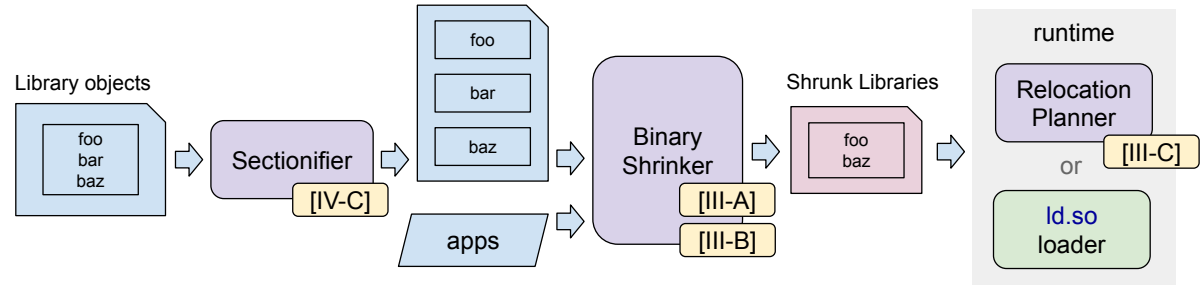


Fig. 2: Overview of LibGLP-1 's workflow. LibGLP-1 takes relocatable library object files and application binaries as input. The sectionifier first converts library object files into a debloating-ready format by placing functions and data into fine-grained sections; this component is described in Section IV-C. The resulting debloating-ready object files, together with the application binary, are then processed by the binary shrinker, which performs static analysis and relinking to generate debloated shared libraries; these steps are described in Sections III-A and III-B. The resulting shrunk libraries can be loaded either by the standard dynamic loader ld.so or, on systems without a dynamic loader, by the relocation planner described in Section III-C.

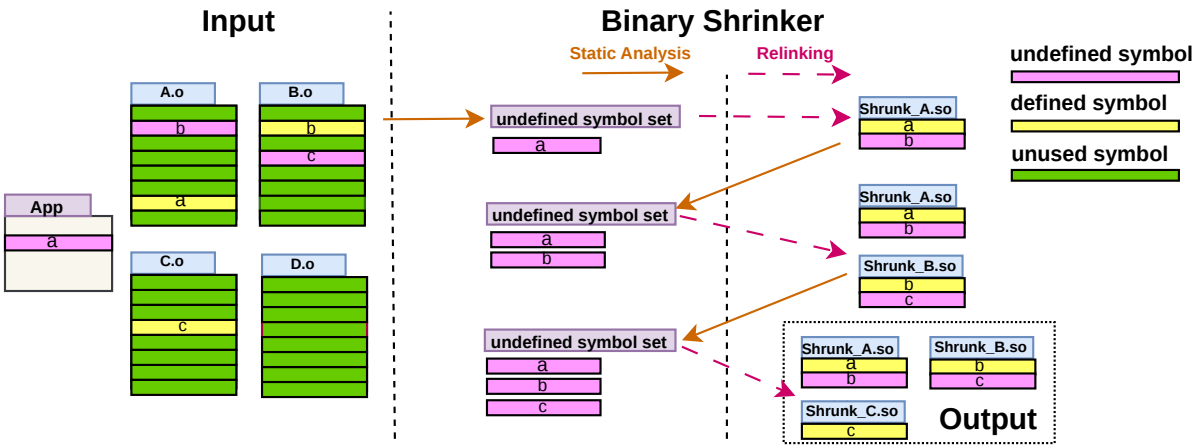


Fig. 3: Overview of the recursive shared-library debloating workflow implemented by LibGLP-1's binary shrinker. The figure illustrates the workflow using a small example with library object files $A.o$, $B.o$, $C.o$, and $D.o$, and symbols a, b, and c. Starting from the application's undefined symbol a, LibGLP-1 performs static analysis to determine which symbols are required from dependent libraries, then relinks a shrunk shared library that retains only those required symbols. In the example, preserving a in Shrunk_A.so exposes a new undefined symbol b, which in turn causes the next iteration to retain b in Shrunk_B.so. This relinking step further exposes symbol c, so the process repeats once more and retains c in Shrunk_C.so. The recursion terminates when no new undefined symbols are introduced, yielding the final set of shrunk shared libraries.

data-flow analysis to understand all possible means of populating the register `%rax` is required. In contrast, LibGLP-1 operates on relocatable ELF objects, which preserve the explicit dependency information needed for static analysis. In particular, symbol table entries identify which functions and data objects are defined or undefined, while relocation entries record which locations in code or data sections refer to those symbols, including references from function-pointer tables or vtables to their targets. We avoid the complexity of binary decompilation and the data-flow analysis as any dependent functions in the previous example as all dependencies that would be the source for `%rax` would appear at the time of access in the symbol table. This allows LibGLP-1 to recover dependencies directly from ELF metadata, avoiding the need for runtime tracing. Based on these explicit dependency relationships, LibGLP-1 can then derive the correct undefined-symbol set for each executable or relinked shared library. Whether this statically derived undefined-symbol set is sufficient for recursive debloating across multiple libraries, however, depends on the following assumptions.

Closed-world assumption. LibGLP-1 assumes a closed-world deployment in which the target executables and their dependent shared libraries are known before deployment. This assumption bounds the scope of dependency analysis to a fixed workload. As a result, LibGLP-1 does not need to account for unknown binaries or libraries outside that workload, and the dependency set recovered statically from the available ELF artifacts is sufficient for workload-specific minimization. This assumption is well matched to embedded software environments, where applications, libraries, and deployment images are usually determined offline and remain stable after deployment. This includes common embedded deployments that adhere to the closed-world assumption [2], [3], [20], [4], [5].

Object-file related assumptions. Similar to [8], we assume that most shared libraries are accompanied by corresponding

static libraries. Further, we assume that the object files in those static libraries retain the metadata required for symbol analysis, including symbol tables and relocation entries. These assumptions are reasonable because shared libraries are commonly distributed together with development artifacts such as static archives, and the ELF symbol and relocation metadata of the object files in those archives is typically preserved for static linking. In our survey, core toolchain runtimes (e.g., `libgcc`), core system libraries (e.g., `libc`), and popular computational or utility libraries (e.g., `libm`) provide the static archives.

B. Relinking

Once the required symbols are derived from the static analysis, LibGLP-1 relinks the objects to not only zero-out the unreferenced code and data, but to also reclaim their memory. The relinking step in LibGLP-1 relies on three elements: relocatable-object inputs, the undefined-symbol set produced by static analysis, and linker garbage collection. Relocatable-object inputs make it possible to generate a new shared-library output by relinking. The undefined-symbol set determines which symbols each shared library must continue to export to satisfy the current workload. During relinking, LibGLP-1 keeps only those required symbols globally visible and localizes the rest. Once unnecessary symbols are no longer conservatively exported, LibGLP-1 leverages linker garbage collection to remove unreferenced sections, allowing the relinked shared library to retain only the required functions and data. As a result, unlike prior systems that provide only coarse-grained removal, such as page-level [7] or object-level elimination [8], LibGLP-1 achieves symbol-granularity removal. To ensure that this removal is simple and robust, LibGLP-1 relies on well-supported linker garbage collection that is already supported by standard linkers. However, linker garbage collection depends on object-file section granularity, and many object files in static archives are not built with the compiler options needed to separate functions and data into fine-grained sections. To address this issue, LibGLP-1 includes a supporting mechanism, discussed in the implementation section, to make such objects amenable to fine-grained garbage collection.

Assumptions. As the output of LibGLP-1's processing is shrunk sets of shared libraries that must contain position-independent code (PIC), we assume that the input static library objects are also PIC. This assumption is reasonable in practice: in our survey, 92 of 96 archives in a typical install of Ubuntu 24.04.2 LTS and 189 of 195 archives on Ubuntu 20.04.6 LTS contain PIC relocatable objects.

C. Runtime-free Deployment

On general-purpose systems such as Linux, the dynamic loader performs symbol resolution and relocation, allowing executables and shared libraries to correctly access undefined symbols at runtime. Many embedded systems, however, do not provide such a loader and therefore cannot automatically resolve symbols or apply relocations for shared libraries. As a result, even if LibGLP-1 successfully debloats a shared library, the shrunk shared library still cannot be deployed directly on those systems. To address this problem, LibGLP-1 shifts symbol resolution and relocation from load time to an offline planning stage.

Before deployment, LibGLP-1 analyzes the executable together with its transitively required shared libraries, resolves symbol references offline, and generates a *relocation plan* that records how each relocation should be patched once concrete base addresses are known.

At load time, instead of performing symbol lookup and relocation, the embedded system only provides the base addresses of the executable and loaded shared libraries, and a lightweight patching routine uses the *relocation plan* to patch the required relocation entries. Overall, LibGLP-1 makes debloated shared libraries deployable on simple embedded systems without a loader through offline relocation planning and lightweight load-time patching.

Discussion. In the embedded setting, base addresses may be fixed ahead of time, which may make load-time patching unnecessary. However, some relocation results still depend on the final deployed memory layout. For example, with Copy Relocation entries, the executable receives its own copy of a global data symbol defined in a shared library at runtime. During offline relocation planning, this symbol is only visible as a definition in the shared library; the executable's own copy does not yet exist. Therefore, load-time patching is required, because offline planning cannot resolve the executable's own copy of the symbol.

IV. Implementation

LibGLP-1 consists of three main components. First, a *binary shrinker* that computes the workload's required symbol set and rebuilds shared libraries to remove unused code and data. Second, a *relocation planner* that precomputes relocation metadata at build time and provides load-time patch APIs for loaders on embedded systems that lack a runtime linker. Third, an *sectionifier* that converts object files into a section-per-function and section-per-data layout.

A. Binary Shrinker

We implement a toolchain that shrinks shared libraries for a given set of executables through iterative static analysis and relinking as we mentioned in the design section. Starting from the executables, the tool first analyzes their undefined symbols, then identifies the shared-library symbols that satisfy those references. It next extracts relocatable object files from the corresponding static archives and relinks a new shared library that retains only the objects needed to provide the current symbol set. Because the relinked library may itself introduce new undefined references to other libraries, the tool reanalyzes the resulting shared libraries, updates the required symbol sets, and repeats relinking. This process continues until the undefined-symbol sets no longer change, yielding a fixed point in which only the symbols and object files required by the executable workload are retained.

Algorithm 1: Recursively Shrinking of Shared Libraries. Function SHRINK iteratively shrinks shared libraries by analyzing undefined symbols of a workload. Initially, it scans the set of executables to identify the required symbols. In subsequent iterations, the inputs are the shrunk shared libraries, ensuring transitive dependencies are covered.

```
1  procedure SHRINK(Libs, ReqSyms);
2    ShrunkLibs ← ∅;
3    foreach L ∈ Libs do;
4      ShrunkSo ← RelinkLibrary(L, ReqSyms);
5      ShrunkLibs ← ShrunkLibs ∪ {ShrunkSo};
6    end for;
7    UndefSyms ← ExtractUndefinedSyms(ShrunkLibs);
8    if UndefSyms = ∅ then;
9      return ShrunkLibs;
10   end if;
11   return SHRINK(Libs, ReqSyms ∪ UndefSyms);
12 end procedure
```

To eliminate unused functions and data from shared libraries during relinking, LibGLP-1 applies linker garbage collection at symbol granularity. However, a challenge arises because shared libraries, by default, retain all symbols as globally visible. This conservative visibility prevents the linker's garbage collection mechanism from removing unused symbols, as the linker must assume that any symbol could be referenced by external programs at runtime.

In closed-world embedded systems, this assumption can be relaxed. Since the complete set of executables and libraries is known in advance, we can determine which symbols are actually needed by the workload. Leveraging this closed world assumption, we generate custom linker "version scripts" that explicitly mark only the required symbols as globally visible, while demoting all other symbols to local visibility. By restricting symbol visibility in this manner, we enable the linker's garbage collection to eliminate unused functions and data from shared libraries.

Linker garbage collection is done on a *section granularity*, thus doesn't automatically work for *symbol* removal. Garbage collection requires object files to be compiled with `-ffunction-sections` and `-fdata-sections`, which place each function and data object in its own section. However, in practice, not all object files are built with these options. To address this, we provide a tool that rewrites such object files into a section-per-function/data layout, enabling fine-grained garbage collection. The overall shrink process is formalized in **Algorithm 1**, a recursive procedure that rebuilds minimal shared libraries by iteratively resolving transitive symbol dependencies across executables and shared libraries. The algorithm terminates by reaching a fixed-point when successive iterations discover no new undefined symbols, ensuring that all required symbol dependencies are fully resolved.

The algorithm operates on the principle that undefined symbols represent the set of required symbols that must be provided by shared libraries. Initially, we collect undefined symbols from the *Dynamic Symbol Table* of all executables, as these directly represent the symbols needed by the workload. As such, the algorithm is initiated with SHRINK(Libs, ExtractUndefinedSyms(Executables)).

The ExtractUndefinedSyms(*binary*) function returns the set of symbols that are undefined (thus must be provided by another library) in the *binary* binary file. The RelinkLibrary(*lib*, *ReqSyms*) links the library *lib* in such a way that includes only symbols in *ReqSyms* that are required by another library or executable and those that are reachable from them. After resolving the required symbols, each library is rebuilt by retaining only these symbols, using version scripts to enforce symbol visibility constraints. For symbol resolution, each undefined symbol is mapped to its providing library, which considers search order, version matching, and symbol aliases. This set of symbols might have undefined symbols referencing other library symbols, which will be resolved on the next pass of the SHRINK function. This proceeds until the algorithm reaches a fixed point when there are no more undefined symbols.

Accurately resolving these symbols throughout this process presents several challenges. We summarize the key challenges and our solutions below.

- **Multiple Definitions**:A multiple-definition case means that the same symbol name is defined in multiple shared libraries. This makes it unclear which shared library should provide that symbol at runtime. To resolve the correct provider, we replicate the standard `ld` symbol resolution algorithm, scanning shared libraries in the order specified by the executable's dependency list. The first matching definition is selected.
- **Symbol Versioning**: Symbol versioning is a mechanism that allows a shared library to provide exported symbols with specific version identifiers. This information must be preserved during relinking to ensure compatibility with existing binaries. However, without access to the source code, we cannot regenerate the shared library's version scripts traditionally used during compilation. Consequently, rebuilt shared libraries lack symbol version information by default. To address this, we recover symbol-versioning metadata from the ELF versioning sections of the original libraries, including the *Version Symbol Table*, *Version Requirements Section*, and *Version Definition Section*, and propagate it to the rebuilt ones.
- **Copy Relocations**: Copy relocations are a dynamic linking mechanism in which a global data symbol defined in a shared library is copied into the executable's writable memory at load time. When copy relocation happens, the executable contains its own runtime copy of that data symbol, even though the original definition comes from the shared library. As a result, since LibGLP-1's dependency analysis mainly follows undefined symbols and symbol references, it may miss the executable's dependency on such shared-library data symbols, because after copy relocation those symbols can appear to be defined locally in the executable. To recover these dependencies, we additionally analyze relocation entries and add copy-

relocated symbols to the set of symbols required by the executable.

- **Symbol Aliases**: Symbol aliases mean that the same function or data symbol can be referred to by different symbol names. When the symbol name referenced by the executable differs from the corresponding symbol name retained in the shared library, LibGLP-1 may incorrectly conclude that no dependency exists. As a result, LibGLP-1 may mistakenly remove a symbol that is actually required by the executable. To handle this, we build alias mappings and, whenever a symbol name is encountered, check whether one of its aliases is used in the shared library, and ensure aliases are preserved during shrinking.

B. Relocation Planner

LibGLP-1 realizes the dynamic loader free deployment of shared objects through a *relocation planner*. This enables embedded systems with a static structure, or with tight memory constraints to avoid requiring a loader. A Python-based *relocation planner* parses each application and its transitively required shared libraries, then emits a compact C relocation plan together with load-time patch APIs.

The relocation plan encodes, for each binary, the locations of unresolved symbol references and the objects that define them. The load-time patch APIs allow the loader to insert the correct addresses once the binaries are mapped in memory, ensuring that all cross-binary symbol references are valid.

Load-time patch APIs:

```
int set_base_addr(const char *exe, const char *so,
    unsigned long base_addr);
int patch_component(const char *exe);
```

The API `set_base_addr` records, in the relocation plan, the runtime load base address of an executable or shared library. The parameter `exe` identifies the executable, while `so` identifies a shared library. The API `patch_component` uses the relocation plan, which records the base addresses and relocation metadata, to process relocation entries and update the actual memory locations where symbol addresses are stored. As such, all reference to symbols between executable and libraries are resolved.

We validate the *relocation planner* on the publicly available Composite OS [21] that constructs a static process structure. Composite statically sets up all programs (components) and their access control at boot-time – similar to Arinc [22], Autosar [23], [24] and camkes [5] – leveraging the *relocation planner* to hook up shrunk shared libraries.

Limitations. Our approach deliberately does not implement late binding or runtime library discovery (*i.e.* no `dlopen`). This is consistent with LibGLP-1's closed-world assumption and with embedded deployments that favor static dependency graphs over dynamic extensibility.

C. Sectionifier

Full support for garbage collection requires object files to be compiled with `-ffunction-sections` and `-fdata-sections`, which place each function and data object in its own section. However, in practice, not all object files are built with these options. To address this, we provide a tool that rewrites such object files into a section-per-function/data layout, enabling fine-grained garbage collection.

Algorithm 2: Split a section into constituent symbols, each with its own section. Symbols are acquired via the symbol table from the `.symtab` section, ordered by value. We then create new section per symbol, and delete the original section.

```
1  procedure SPLIT(ObjectFile, OrigSect);
2    Syms ← GetSymbols(ObjectFile, OrigSect);
3    foreach (Sym, SymEnd) ∈ Syms do;
4      SectData ← GetSectData(OrigSect, Sym, SymEnd);
5      Sect ← AddSection(ObjectFile, SectData);
6      AddSymbol(ObjectFile, Sect, Sym);
7      RemapRelocation(ObjectFile, Sect, Sym);
8    end for;
9    DeleteSection(ObjectFile, OrigSect);
10 end procedure
```

We implement the *sectionifier* using ELFIO, a C++ library for binary file manipulation [25]. This tool replicated the effects of the aforementioned compilation without relying on source code. We parse the object files to find sections such as `.text`, `.data`, and `.rodata`, scan the symbol table, and generate a section for each symbol. The steps are summarized in **Algorithm 2**, wherein the function SPLIT(ObjectFile, OriginalSection) is called for each section that needs partitioning symbols into separate sections. We retrieve the section's symbol list Symsfrom the symbol table via the `.symtab` section, and order the symbols by their value (offset from the corresponding section's offset).

For each symbol, the GetSectData(OrigSect, Sym, SymEnd) function retrieves the symbol's data by slicing the original section's data from the symbol's offset to the next symbol's offset. A new section is then created with this data via AddSection(ObjectFile, SectData), and the symbols corresponding to the function and section is added to the symbol table with AddSymbol(ObjectFile, Sect, Sym). When this section is created, all relocation entries still reference and refer to the symbol in the previous section. These relocations must be remapped to point to the new section and symbol, which is done by the RemapRelocation(ObjectFile, Sect, Sym) function. The same function also copies relocation entries to a new split relocation section, such as from `.rela.text` to `.rela.text.fn`.

The `sectionifier` enables debloating of `openSSL`, a popular ssl client library, whoose shared dependencies are not built with `-ffunction-sections` and `-fdata-sections` by default. We take `libcrypto.so` and `libssl.so` from `openSSL` as input and applied `sectionifier` to create new versions of these libraries with a section-per-function/data layout. We verified that the resulting libraries are functionally equivalent to the original ones by running an vsftpd server which uses openssl [26], and that they enable effective garbage collection when used in our *binary shrinker*.

Note that while the `sectionifier` does process library objects, it does *not* decompile the code. This maintains LibGLP-1's symbol-centric approach processing on ELF binaries.

V. Evaluation

In this section, we provide a comprehensive evaluation of LibGLP-1's capability to debloat shared libraries. In Subsection V-A, we present a single-application case study to evaluate whether LibGLP-1 can effectively debloat the shared libraries required by a concrete application while preserving correctness. In Subsection V-B, we present a whole-system evaluation to examine whether LibGLP-1 remains effective when multiple applications and shared libraries interact at system scale. In Subsections V-C, V-D, and V-E, we compare LibGLP-1 with prior work and highlight how it differs from representative existing debloating approaches. Our primary evaluation metrics are on-disk file size and mapped-memory footprint, and we further validate correctness by executing the target applications with the debloated library set.

A. *librealsense* Debloating

We use `librealsense` [27] as the case study for debloating single applications. This library is representative of a common pattern in robotics and autonomous embedded systems: a feature-rich shared library that supports many devices and capabilities, but deployed applications typically use only a small subset of that functionality.

Our evaluation platform runs Ubuntu 20.04.6 LTS on Intel Core i5-6400 CPU and 8GB of RAM. The sensor used in the experiment is Intel RealSense Tracking Camera T265, a robotics-oriented device that provides visual-inertial tracking and motion estimation for localization and navigation tasks.

As the target workload, we use rs-pose v1.0, an application built on top of `librealsense` to interface with the Intel T265 IMU, stream pose data from the sensor, and feed that data to control algorithms running on an autonomous rover.

The version of `librealsense` used in our experiments is 2.51, which runs stably on Ubuntu 20.04.6 LTS. We evaluate whether LibGLP-1 can shrink `librealsense` and the shared libraries transitively required by `rs-pose` while preserving the correct operation of `rs-pose`. We report the resulting reductions in on-disk library size in Figure 4 and memory footprint in Figure 5. Both figures demonstrate the effectiveness of LibGLP-1 for shared-library debloating, with substantial reductions in both on-disk size and memory footprint. We reran `rs-pose` with the complete debloated library set and observed correct execution. Notably, the memory footprint of `librealsense` reduced from 40 MB to 28 MB, saving about 12 MB. Its on-disk size reduced from 170 MB to 40 MB, saving 130 MB. These reductions are particularly significant for resource-constrained embedded systems.

Discussion. In both figures the shrunk `libcap` is shown as 0 because it is no longer required in the transitive dependency closure after debloating. This demonstrates that LibGLP-1 can eliminate unnecessary dependencies entirely, rather than merely reducing their size. The reduction in on-disk size is more substantial because the original libraries were either taken directly from their upstream repositories or built using the default configuration, both of which preserve debug sections, whereas the debloated libraries do not retain them.

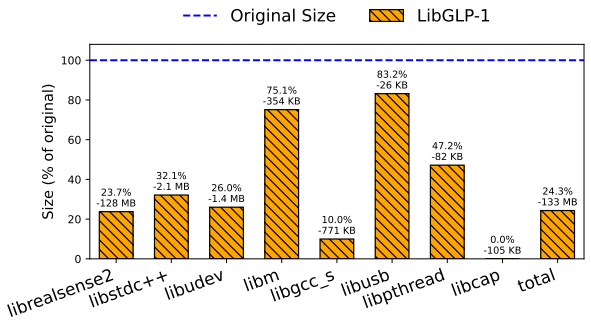


Fig. 4: Normalized on-disk file size of `librealsense` and all shared libraries transitively required by `rs-pose`, together with their debloated versions produced by LibGLP-1.

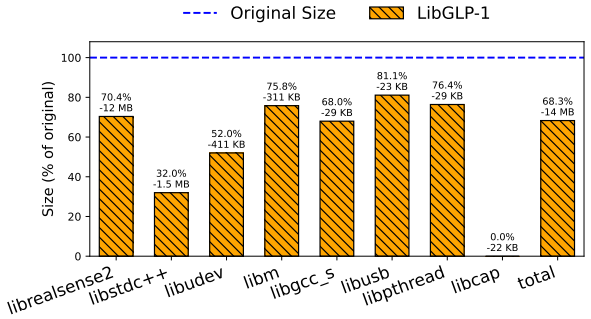


Fig. 5: Normalized memory footprint of `librealsense` and all shared libraries transitively required by `rs-pose`, together with their debloated versions produced by LibGLP-1.

B. Microkernel OS Debloating

We evaluate LibGLP-1 's whole-system debloating capability in a component-based microkernel OS environment, which is common in safety-critical and resource-constrained embedded systems. We choose Composite as the target microkernel OS because its component-based design makes system dependencies explicit and provides a concrete deployment setting for evaluating both LibGLP-1 's whole-system debloating capability and its relocation planner. Composite boots up into statically-configured protection domains, thus avoids the complexity of a dynamic loader. As such, it uses the *relocation planner* to load shared libraries into each process, and the amount of code added to the Composite boot-time initialization to integrate with the *relocation planner* is 13 lines of code. The full system runs in QEMU 6.2.0 on Ubuntu 22.04.3 LTS and is compiled with GCC 11.4.0.

Our experiments run four ML applications from the SOD embedded framework on Composite. Together, these applications correspond to common stages in an embedded vision pipeline: image resize, image rotation, edge detection, and license plate detection. This selection covers diverse SOD functionality and dependency patterns, ensuring that LibGLP-1 is evaluated on a workload that links against computer-vision/ML, networking, and core OS-service libraries.

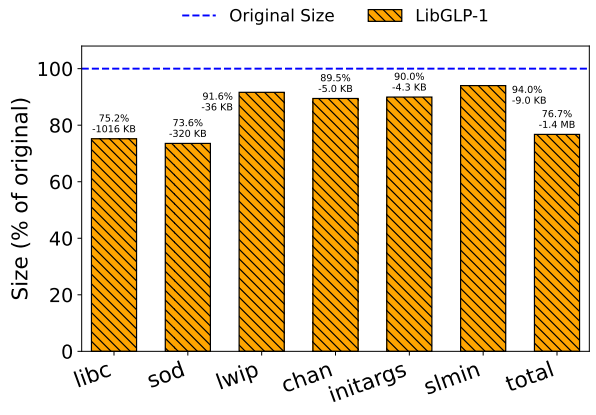


Fig. 6: Normalized on-disk file size of all shared libraries in Composite, together with their debloated versions produced by LibGLP-1.

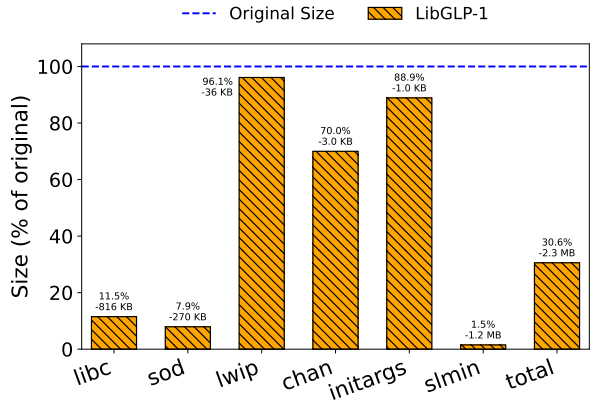


Fig. 7: Normalized memory footprint of all shared libraries in Composite, together with their debloated versions produced by LibGLP-1.

These applications use the Patina RTOS APIs [28] to request OS services and dynamically link against the following shared libraries: `sod v1.1.8`, `lwIP v2.1.4`, `musl libc v1.2.0`, and the `chan`, `initargs`, and `slmin` libraries. `sod` is an embedded computer vision and machine learning library. `lwIP` is a lightweight TCP/IP stack. `musl libc` is the musl version of C standard library. The remaining libraries provide core Composite services: `chan` supports channel-based communication, `slmin` supports component scheduling, and `initargs` supports component initialization.

Figure 6 shows the on-disk file-size reduction of these shared libraries. Figure 7 summarizes memory savings of these shared libraries.

Discussion. `lwIP` exhibits only little savings in our evaluation. The lwIP stack provides configurable memory pools and protocol features, and its memory footprint is highly dependent on the selected configuration. In our build, many memory pools and control structures remain present in the shrunk library due to the selected configuration, which limits the opportunities for reduction. `chan` and `initargs` are small libraries that provide specialized OS services rather than general-purpose functionality. As a result, most of their code and data are required by the Composite OS, leaving few opportunities for debloating. In contrast, `sod` and `libc` are feature-rich, general-purpose libraries, and a given workload typically exercises only a subset of their functionality. Consequently, LibGLP-1 can remove substantial unused code and data, resulting in large reductions. Additionally, `slmin` highlights the benefit of removing unused `.data/.bss` section. In our workload, a fixed-size memory pool (`_blkpts`) in library `slmin` is unused and eliminated, reducing mapped memory by over 1000 KB, which is substantial for resource-constrained embedded systems.

C. Comparision with D-Linker

We evaluate LibGLP-1 against D-Linker, a representative prior tool for object-level debloating of shared libraries and, to the best of our knowledge, the first tool in this space. D-Linker debloats a shared library by retaining required ELF object files and relinking the library from those retained objects. This comparison helps clarify what symbol-level debloating gains over object-level debloating. The main difference between our approach and D-Linker lies in the removal granularity. Our current implementation performs debloating at the granularity of individual function and data symbols, while D-Linker debloats at the granularity of ELF object files. In detail, D-Linker identifies the required object files of a shared library and relinks only those retained object files. As a result, D-Linker naturally preserves all code and data contained in every retained object file, including unused functions or data symbols that happen to reside in those objects. Moreover, once an object file is retained because it provides a required symbol, additional object files may also need to be kept to satisfy transitive dependencies.

Because the source code and implementation of D-Linker are unavailable, we do not directly reproduce their tool. Instead, we approximate D-Linker's object-level model by restricting removal granularity to object files and relinking the library from the retained object files, rather than performing finer-grained function/data elimination.

For the evaluation, we use `vsftpd` 3.0.3 as the target application, since it is widely used in prior debloating studies, together with its dependent shared libraries: `libcrypto` and `libssl` from OpenSSL 3.0.13, `libcrypt` 2.0.0, and `musl libc` 1.2.2.

Figure 8 and Figure 9 show that symbol-level debloating consistently achieves greater shared-library memory savings than object-level debloating, as expected, because object-level debloating must retain an entire object file whenever any symbol in it is needed, thereby conservatively preserving other unused code and data in the same object file.

Discussion. Figure 9 indicates two special cases: `libcrypt` shows almost no reduction under either method, while `musl-libc` exhibits a similar degree of reduction under both methods. Inspection of `libcrypt` shows that the public `crypt` interfaces transitively reach nearly all object files in the archive, leaving little opportunity for object-level removal. Moreover, the retained code and data remain densely connected, which further limits the effectiveness of symbol-level

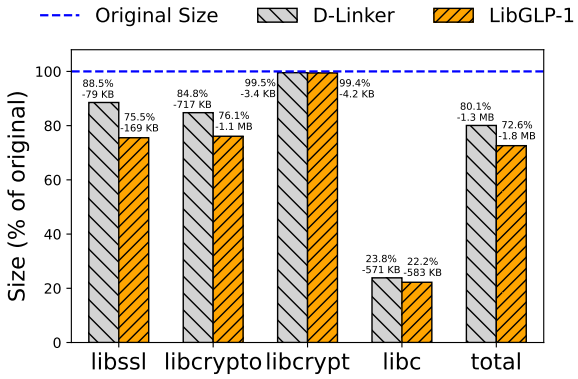


Fig. 8: On-disk file size comparison between object-level debloating and LibGLP-1's symbol-level debloating for `vsftpd` and its dependent shared libraries. All values are normalized to the original shared libraries as the baseline.

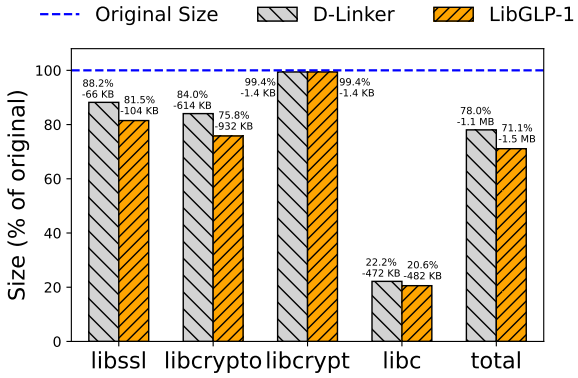


Fig. 9: Memory size comparison between object-level debloating and LibGLP-1's symbol-level debloating for `vsftpd` and its dependent shared libraries. All values are normalized to the original shared libraries as the baseline.

debloating. Consequently, both object-level and symbol-level debloating only limited reduction on `libcrypt`. Inspection of `libc` shows that object-level debloating achieves results close to our symbol-level debloating on musl libc, because musl libc already uses a relatively fine-grained object layout: many object files contain only a single function, which reduces the gap between object-level and symbol-level removal.

D. Comparision with ELFTailor

We evaluate LibGLP-1 against ELFTailor, a representative prior tool for binary-level debloating of shared libraries. ELFTailor debloats shared libraries by identifying unused functionality in existing binaries and overwriting the corresponding code directly in the binary image. This comparison helps clarify what symbol-level debloating gains over binary-level debloating. Our comparison is based on the ELFTailor source project released by the authors [7]. All experiments are conducted on Ubuntu 22.04.3 LTS. To reproduce ELFTailor, we use `pyelftools` 0.26, `tqdm` 4.40.0, `six` 1.12.0, and `jq` 1.6.0. Similar to D-Linker, our primary focus is on comparing the memory-reduction effectiveness of ELFTailor and LibGLP-1. For evaluation, we use `vsftpd` 3.0.3 as the target application and `musl libc` 1.2.2 as the target C library.

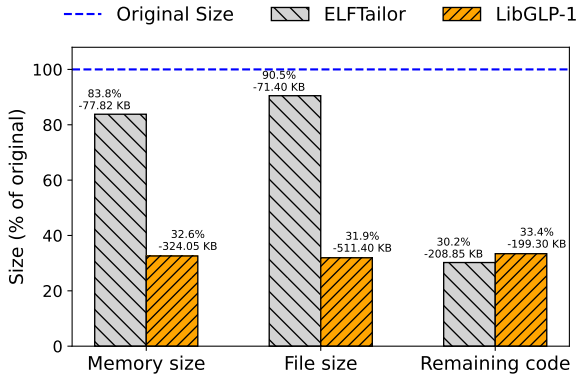


Fig. 10: Comparison between the ELFTailor-debloated and LibGLP-1-debloated musl libc. All values are normalized to the original musl-libc baseline. "Remaining code" denotes the non-overwritten `.text` bytes for ELFTailor and the retained `.text` size for LibGLP-1.

ELFTailor operates in four stages. First, it performs static analysis to recover inter-library dependencies and construct a call graph. Second, it refines this result through dynamic tracing based on uprobes. Third, it removes unused functions by deleting the corresponding symbol metadata and overwriting unused code with trap instructions. Finally, it applies a post-processing stage to shrink the ELF layout.

To support ELFTailor's dynamic tracing stage, we design a scripted end-to-end FTP workload for `vsftpd`. The workload launches `vsftpd` with the target libraries through the musl dynamic loader, connects to the server at `127.0.0.1:2121`, creates a directory, uploads a file, verifies that the file is stored correctly, reconnects to delete the file, removes the directory, and then terminates the server. This workload exercises server startup, FTP session establishment, directory operations, file upload, file deletion, and shutdown.

Figure 10 shows the debloating effectiveness of ELFTailor and LibGLP-1 in terms of file size, memory size, and remaining code. ELFTailor reduces file size and memory size to 83.4% and 83.8% of the original library, respectively, showing that its post-processing stage is able to save some storage and mapped-memory. However, since ELFTailor can only remove the whole pages, in this case, 18 of 72 pages are fully overwritten,the reduction is not huge as expected. LibGLP-1 achieves larger reductions, lowering file size to 31.3% of the baseline and memory size to 32.6%. For remaining code, ELFTailor leaves 30.2% of the original code bytes, while LibGLP-1 retains 33.4%.

Discussion. Figure 10 also shows that ELFTailor achieves a smaller amount of remaining code than LibGLP-1. One reason is consistent with the observation made in the D-Linker paper: ELFTailor performs finer-grained binary analysis and removes unused functionality by overwriting dead function bodies, which reduces the amount of code left in the `.text` region. Another reason is specific to musl libc, which packages both the libc implementation and the dynamic loader in the same shared object. Because this violates our assumption

that a target shared library only provides functionality to the application, our approach conservatively preserves dynamic-loader-related code. ELFTailor, by contrast, supports a manually specified blacklist that protects critical symbols from removal. With this manual assistance, ELFTailor can still overwrite other unused functions in the dynamic loader.

E. Comparison with Piece-Wise Tool

We evaluate LibGLP-1 against Piece-Wise, a representative prior tool for source-level debloating of shared libraries. Piece-Wise relies on compiler analysis to identify unused functions and on a runtime loader to disable them. This comparison helps clarify what symbol-level debloating gains over source-level debloating. Since the Docker linker for the Piece-Wise toolchain provided in its github repository is no longer available, we reproduced the full Piece-Wise pipeline from the source code in the same repository by creating a new Docker environment with docker v27.5.1, Ubuntu v16.04, LLVM v4.0, musl-libc v1.1.15, and coreutils v8.2.

We measure the runtime performance of each program in coreutils v8.2 built with the Piece-Wise toolchain and with LibGLP-1. Note that the two toolchains necessarily use different compilers: Piece-Wise uses LLVM v4.0 as required by its pipeline, whereas LibGLP-1 uses the host GCC toolchain (musl-gcc v5.4.0) because LLVM v4.0 does not support the garbage-collection mechanism needed by LibGLP-1. We report this difference for transparency and use the same OS, docker, musl version, and coreutils version to keep the rest of the evaluation conditions consistent.

Because the Piece-Wise loader executes at runtime, the performance we measure is approximately 40× slower than the native programs. In contrast, LibGLP-1 does not insert additional runtime components, so its performance is close to the native programs.

Regarding memory, LibGLP-1 reduces musl's `libc.so` by roughly 50% (saving 488 KB of mapped memory), whereas the Piece-Wise configuration increases `libc.so` by about 5.7× compared to the original. Details are shown in Table I.

TABLE I: `libc` footprint comparison: file size and mapped memory.

Library	Original (KB)		LibGLP-1 (KB)		Piece-Wise (KB)	
	File	Mem	File	Mem	File	Mem
`libc`	895	902	416	424	5084	5096

Because Piece-Wise primarily targets attack-surface reduction and reports gadget counts for `coreutils` 8.2 using `ROPgadget`, we also measure gadget-count reduction on `coreutils` 8.2 using the same tool. Since the Piece-Wise paper does not specify which version of `ROPgadget` it used, we use `ROPgadget` v7.7 in our experiments [29]. `ROPgadget` is a static analysis tool that scans executable code in a binary and uses the Capstone disassembler to find candidate gadget instruction sequences. It reports multiple classes of candidate gadgets, such as return-oriented programming (ROP), jump-oriented programming (JOP), and syscall-oriented (SYS) gadgets. In our evaluation, we run `ROPgadget` on each generated binary and compute gadget-count reduction from the aggregate numbers of reported gadgets before and after debloating. We use this reduction number only as a comparative metric, not as a strict proof of security.

The results are shown in Table II. The Piece-Wise github release does not include the pointer-analysis stage described in the Piece-Wise paper, which likely contributes to the lower gadget reductions observed for our source-built version.

TABLE II: ROP gadget reduction across 101 `coreutils`: LibGLP-1 vs. Piece-Wise (source-built) vs. Piece-Wise (paper-reported).

Method	Min (%)	Mean (%)	Max (%)
LibGLP-1	78.98	83.76	85.66
Piece-Wise (source-built)	55.43	59.60	61.28
Piece-Wise (paper-reported)	64.92	71.69	77.33

A natural concern is that a single debloated shared library used by many programs must retain the union of all functions required by any program, which can limit the achievable ROPgadget reduction. To address this concern, similar to Piece-Wise, LibGLP-1 supports per-program debloating. This design exposes an explicit trade-off: designers can deploy per-program shrunk libraries to maximize ROP-gadget reduction, or allow multiple programs to share a single shrunk library to improve memory savings at the cost of smaller reductions.

VI. Related Work

A plethora of avenues can benefit from software debloating [10]. Debloating approaches cover various domains, from dynamic libraries [7], [15], [8] to OS kernels [13]. The approaches listed below are the closest match to our work as they try to debloat shared libraries in one form or another.

Piece-wise [15] is motivated by security and focuses on reducing exposed code. It relies on source code and a custom LLVM-based toolchain: the compiler emits per-function dependency metadata into the generated binary, and a custom loader consumes this metadata at runtime, so only the required functions remain available. Rather than performing pruning in the loader, LibGLP-1 shrinks shared libraries offline.

ELFtailor [7] shrinks binaries via binary-level analysis and rewriting. It disables unused functions by rewriting their code regions and modifying ELF structures to reduce the footprint. A key limitation is page-granularity compaction: it typically reclaims space only when an entire page becomes unused. Similar, Nibbler [19] is another binary-level shared-library debloating tool. It analyzes binaries and shared libraries, reconstructs a conservative function-call graph, and erases unreachable library functions. In contrast, LibGLP-1 minimizes libraries at symbol-level granularity, enabling more fine-grained debloating.

D-Linker [8] debloats shared libraries by relinking them from object files. It rebuilds dependency requirements and rebuilds libraries—keeping only the needed .o objects. It also supports dynamic linking mechanisms to further remove object files. However, its removal granularity is limited to object-file level. In contrast, LibGLP-1 removes unused functions and data at symbol level rather than at object-file level.

uTrimmer [17] debloats shared libraries at the basic block level. However, the scope of this tool is only limited to binaries that run on MIPS architecture, severely limiting its

application. Furthermore, uTrimmer only debloats code, leaving data sections bloated. Similar to uTrimmer, MiniLib[30] and Trimming[18] are limited in scope, as they only debloat libraries written in Java. In contrast, LibGLP-1 supports the x86-64 architecture and libraries written in any language. .

Software Multiplexing [9] merges a predetermined set of applications and their libraries at the LLVM IR level into a single statically linked executable, enabling cross-application optimization. However, when multiple executables are built for different application sets with overlapping applications or libraries, duplicated code can persist across bundles. In contrast, LibGLP-1 does not require LLVM IR or source-level whole-program bundling, and it does not suffer from the same cross-bundle duplication limitation.

VII. Conclusions

This paper introduces LibGLP-1, which provides a symbol-level mechanism for debloating shared libraries. It utilizes a *binary shrinker* that performs static analysis over application and library ELF objects to garbage collect all unused symbols. The garbage collection mechanisms require that all symbols are in separate object sections, a constraint we address with the *sectionifier*. Unlike existing systems, LibGLP-1 depends only on ELF objects, avoiding the need for error-prone decompilation techniques or source code. Additionally, we simplify the deployment of shared objects in embedded systems with a static structure by opting for a relocation planner instead of a complex loader. Despite being a static approach operating at the ELF level, LibGLP-1 consistently achieves greater reduction than the state of the art.

References

[1] "Mobile dram industry analysis. https://www.trendforce.com/research/category/Semiconductors/DRAM."

[2] "Yocto project documentation. https://docs.yoctoproject.org/."

[3] "The buildroot user manual. https://buildroot.org/downloads/manual/manual.html."

[4] "The Composite component-based system: http://composite.seas.gwu.edu."

[5] I. Kuz, Y. Liu, I. Gorton, and G. Heiser, "Camkes: A component model for secure microkernel-based embedded systems," *Journal of Systems and Software*, vol. 80, no. 5, pp. 687–699, 2007.

[6] "Docker: https://www.docker.com/," 2018.

[7] A. Ziegler, J. Geus, B. Heinloth, T. Hönig, and D. Lohmann, "Honey, i shrunk the elfs: Lightweight binary tailoring of shared libraries," *ACM Trans. Embed. Comput. Syst.*, vol. 18, no. 5s, Oct. 2019. [Online]. Available: https://doi.org/10.1145/3358222

[8] J. He, P. Hou, J. Yu, J. Qi, Y. Sun, L. Li, R. Zhao, and Y. Wu, "D-linker: Debloating shared libraries by relinking from object files," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 11, pp. 3768–3779, 2024.

[9] W. Dietz and V. Adve, "Software multiplexing: share your libraries and statically link them too," *Proc. ACM Program. Lang.*, vol. 2, no. OOPSLA, Oct. 2018. [Online]. Available: https://doi.org/10.1145/3276524

[10] M. D. Brown, A. Meily, B. Fairservice, A. Sood, J. Dorn, E. Kilmer, and R. Eytchison, "A broad comparative evaluation of software debloating tools," in *33rd USENIX Security Symposium (USENIX Security 24)*. Philadelphia, PA: USENIX Association, Aug. 2024, pp. 3927–3943. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/brown

[11] H.-C. Kuo, J. Chen, S. Mohan, and T. Xu, "Set the configuration for the heart of the os: On the practicality of operating system kernel debloating," *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 4, no. 1, May 2020. [Online]. Available: https://doi.org/10.1145/3379469

[12] V. Rastogi, D. Davidson, L. De Carli, S. Jha, and P. McDaniel, "Cimplifier: automatically debloating containers," in *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*, ser. ESEC/FSE 2017. New York, NY, USA: Association for Computing Machinery, 2017, p. 476–486. [Online]. Available: https://doi.org/10.1145/3106237.3106271

[13] H.-C. Kuo, D. Williams, R. Koller, and S. Mohan, "A linux in unikernel clothing," in *Proceedings of the Fifteenth European Conference on Computer Systems*, ser. EuroSys '20. New York, NY, USA: Association for Computing Machinery, 2020. [Online]. Available: https://doi.org/10.1145/3342195.3387526

[14] Z. Zhang, W. You, G. Tao, G. Wei, Y. Kwon, and X. Zhang, "Bda: practical dependence analysis for binary executables by unbiased whole-program path sampling and per-path abstract interpretation," *Proc. ACM Program. Lang.*, vol. 3, no. OOPSLA, Oct. 2019. [Online]. Available: https://doi.org/10.1145/3360563

[15] A. Quach, A. Prakash, and L. Yan, "Debloating software through Piece-Wise compilation and loading," in *27th USENIX Security Symposium (USENIX Security 18)*. Baltimore, MD: USENIX Association, Aug. 2018, pp. 869–886. [Online]. Available: https://www.usenix.org/conference/usenixsecurity18/presentation/quach

[16] L. Dramko, J. Lacomis, E. J. Schwartz, B. Vasilescu, and C. Le Goues, "A taxonomy of c decompiler fidelity issues," in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 379–396.

[17] H. Zhang, M. Ren, Y. Lei, and J. Ming, "One size does not fit all: security hardening of mips embedded systems via static binary debloating for shared libraries," in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 255–270. [Online]. Available: https://doi.org/10.1145/3503222.3507768

[18] X. Song, H. Yu, Y. Wang, and Z. Zhu, "Debloating software through enhanced static analysis and constraint rules," *Software: Practice and Experience*, vol. 55, no. 10, pp. 1696–1710, 2025. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/spe.70003

[19] I. Agadakos, N. Demarinis, D. Jin, K. Williams-King, J. Alfajardo, B. Shteinfeld, D. Williams-King, V. P. Kemerlis, and G. Portokalidis, "Large-scale debloating of binary shared libraries," *Digital Threats*, vol. 1, no. 4, Dec. 2020. [Online]. Available: https://doi.org/10.1145/3414997

[20] "The openwrt developers. 2004. openwrt, a highly extensible gnu/linux distribution for embedded devices. https://openwrt.org/."

[21] Q. Wang, Y. Ren, M. Scaperoth, and G. Parmer, "Speck: A kernel for scalable predictability," in *Proceedings of the 21st IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 2015.

[22] J. Windsor and K. Hjortnaes, "Time and space partitioning in spacecraft avionics," *Space Mission Challenges for Information Technology, IEEE International Conference on*, vol. 0, pp. 13–20, 2009.

[23] "Automotive Open System Architecture (AUTOSAR) Classic Platform. https://www.autosar.org/standards/classic-platform."

[24] S. Fürst and M. Bechter, "Autosar for connected and autonomous vehicles: The autosar adaptive platform," in *2016 46th annual IEEE/IFIP international conference on Dependable Systems and Networks Workshop (DSN-W)*. IEEE, 2016, pp. 215–217.

[25] "Elfio: Elf object and executable file reader/writer. https://github.com/serge1/ELFIO."

[26] vsftpd Project, "vsftpd: Very secure ftp daemon," https://security.appspot.com/vsftpd.html, accessed: 2026-03-30.

[27] Intel RealSense, "librealsense: Intel realsense sdk," https://github.com/realsenseai/librealsense, 2026, accessed: 2026-03-29.

[28] S. Jero, J. Furgala, R. Pan, P. K. Gadepalli, A. Clifford, B. Ye, R. Khazan, B. C. Ward, G. Parmer, and R. Skowyra, "Practical principle of least privilege for secure embedded systems," in *IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 2021.

[29] J. Salwan, "Ropgadget," https://github.com/JonathanSalwan/ROPgadget, 2026, accessed: 2026-03-29.

[30] L. Kopanias, P. Sotiropoulos, N. Kolokotronis, and C. Vassilakis, "Minilib: A flow analysis–based approach for attack surface reduction through software debloating," in *2025 IEEE International Conference on Cyber Security and Resilience (CSR)*, 2025, pp. 260–266.